

CloudMart — Capstone Project Brief

AWS-Native Serverless E-Commerce Backend

Duration: 40 Calendar Days (approx. 30 Working Days) | **Start Date:** 11 August 2026 | **End Date:** 18 September 2026 | **Evaluation Period:** 21 September – 30 September 2026

Language: Python | **IaC:** CloudFormation | **CI/CD:** GitHub Actions | **VCS:** GitHub

Objective and Business Context

Cloud-native development is no longer a differentiator — it is the baseline expectation for backend engineers working in modern product companies. The ability to design, build, secure, deploy, and monitor a distributed system on AWS is a core competency that goes beyond knowing individual services in isolation.

This project bridges the gap between learning AWS services one at a time and applying them together in a coherent, production-representative system. Participants will experience the full engineering lifecycle — from blank-page architecture design to a running, observable, continuously deployed system — making the same kinds of decisions a senior engineer would make on a real team.

The project deliberately mirrors real production engineering practices: infrastructure as code, authentication on every API endpoint, automated deployment pipelines, event-driven architecture, and operational observability. There is no prescribed architecture diagram to copy. Participants must reason about tradeoffs, justify their choices, and own every decision from VPC CIDR ranges to DynamoDB key design.

Problem Statement / Use Case

CloudMart is a lightweight e-commerce backend platform. A growing online retailer needs a backend system to manage their product catalog, process customer orders, track inventory in real time, and give their operations team visibility into daily sales activity.

The system must handle:

- A product team that needs to create, update, and query a product catalog
- Customers placing orders through a mobile or web frontend (the frontend does not exist — participants build only the backend API)

- All API endpoints protected by token-based authentication — unauthenticated requests are rejected before reaching application logic
- Automatic inventory deduction when orders are confirmed, with alerts when stock runs low
- An operations team that needs a daily sales report and a live dashboard showing system health
- Reliable order processing that does not lose data even when individual components fail
- The entire system deployed and updated exclusively through a GitHub Actions CI/CD pipeline — no manual AWS console deployments after the initial bootstrap
- The ability for the entire system to be torn down and redeployed from scratch by running the pipeline — repeatability is a first-class requirement

The system does not need to handle payments or multi-region deployment.

Scope and Expected Deliverables

In Scope

Authentication Layer

- A Lambda Authorizer function attached to all API Gateway endpoints
- The authorizer validates a `Bearer` token from the `Authorization` header against a SecureString stored in SSM Parameter Store
- Valid tokens receive an `Allow` IAM policy; invalid or missing tokens receive a `Deny` — returning `401` to the caller
- Authorizer cache TTL configured to avoid calling SSM on every request
- Token rotation procedure documented in the runbook

Service Domain 1 — Product & Inventory Service

- REST API to create, read, update, and delete products — all endpoints protected by the Lambda Authorizer
- Product data (name, description, price, category, stock count) stored in RDS MySQL
- Inventory update events published to EventBridge when stock changes
- SNS alert when any product's stock falls below a configurable threshold stored in SSM Parameter Store

Service Domain 2 — Order Processing Service

- REST API to place and retrieve orders — all endpoints protected by the Lambda Authorizer

- Orders stored in DynamoDB with access patterns for lookup by order ID and by customer
- Order placement triggers an event-driven workflow: validate stock availability → deduct inventory in RDS → persist order in DynamoDB → send confirmation notification via SNS
- Failed orders routed to an SQS Dead Letter Queue
- All order lifecycle events published to EventBridge

Service Domain 3 — Reporting & Operations Service

- EventBridge scheduled rule triggers a Lambda daily to aggregate order stats and write a CSV report to S3
- A t2.micro EC2 instance in a public subnet runs a lightweight Python Flask app that reads report data from S3 and summary stats from RDS, serving an internal HTML dashboard

Dashboards, Alarms, and Metrics

- A CloudWatch dashboard named `cloudmart-operations` containing the following widgets:
 - API Gateway: total request count, 4xx error rate, 5xx error rate
 - Lambda: invocation count, error count, and duration (p50, p90) for each function
 - SQS: number of messages visible in the orders queue and DLQ depth
 - RDS: CPU utilization, database connections, and free storage space
 - Custom business metrics: `OrdersPlaced`, `OrdersFailed`, `LowStockEvents` — published from Lambda using the `cloudmart` namespace
- CloudWatch alarms with SNS delivery for: Lambda error rate exceeds threshold, DLQ message count > zero, RDS CPU > 70%, API Gateway 5xx error rate exceeds threshold
- All alarms and dashboard defined in the `monitoring-stack` CloudFormation template — not created manually in the console

Infrastructure as Code

- Custom VPC with public and private subnets across one availability zone
- RDS in a private subnet — no direct internet access
- All infrastructure defined in CloudFormation, organized into separate parameterized stacks with cross-stack references
- Lambda deployment packages stored in S3 and referenced from CloudFormation
- All configuration managed via SSM Parameter Store — no hardcoded values in code or templates
- No customer-managed KMS keys — AWS-managed keys used for SSM SecureString, RDS encryption at rest, and S3 SSE

CI/CD Pipeline (GitHub Actions)

- A GitHub Actions workflow triggers on every push to `main`
- Pipeline steps: lint Python code → package Lambda functions → upload packages to S3 → deploy/update CloudFormation stacks in dependency order → run a smoke test against the deployed API
- Stack updates use CloudFormation Change Sets — no direct stack updates
- AWS credentials stored as GitHub Actions secrets (OIDC-based or access key — participant chooses and justifies)
- Separate workflow for infrastructure-only changes vs application-only changes

IAM and Security Standards

- IAM roles with least-privilege policies — no wildcard actions or resources
- RDS security group allows inbound MySQL only from Lambda security group — not from the internet
- S3 buckets block all public access
- No credentials, ARNs, or account IDs hardcoded in any file committed to GitHub

Out of Scope

- Frontend or UI beyond the EC2 Flask dashboard
- Cognito or third-party identity providers
- Multi-region or multi-AZ deployment
- Payment processing
- Load testing

Expected Deliverables

#	Deliverable	Description
1	GitHub Repository	All CloudFormation templates, Lambda source code, GitHub Actions workflows, EC2 Flask app — organized with meaningful commit history throughout the project
2	Architecture Diagram	draw.io or equivalent — all services, data flows, VPC layout, security boundaries, and the CI/CD pipeline flow
3	Data Model Document	RDS entity-relationship diagram with table definitions and indexes; DynamoDB access pattern analysis with partition key, sort key, and GSI justification

#	Deliverable	Description
4	Deployment Runbook	Step-by-step instructions for deploying the entire system from zero into a fresh AWS account, including GitHub Actions secrets setup, stack creation order, and teardown steps
5	Walkthrough Demo	20-minute live demonstration covering: architecture decisions, a working authenticated API call end-to-end, the CI/CD pipeline triggering on a code push, and the CloudWatch dashboard live

Technologies and Concepts to Be Applied

AWS Services

Service	Purpose in Project
VPC	Custom network isolation — public subnet for EC2, private subnet for RDS; security groups as per-resource firewalls
IAM	Per-service execution roles, least-privilege policies, OIDC trust for GitHub Actions
CloudFormation	All infrastructure as code — parameterized stacks, cross-stack references, Change Set-based updates
API Gateway	REST APIs for Product and Order services, request body validation, Lambda Authorizer, method-level throttling
Lambda	All serverless compute — authorizer, product handler, order handler, inventory checker, order processor, report generator
RDS MySQL	Relational storage for product catalog and inventory — normalized schema, private subnet, accessed via Lambda inside VPC
DynamoDB	NoSQL storage for orders — access-pattern-first key design, GSI for customer lookup
SQS	Asynchronous order processing queue, Dead Letter Queue for failed orders
SNS	Low-stock alerts, order confirmation notifications, CloudWatch alarm delivery

Service	Purpose in Project
EventBridge	Order lifecycle events, inventory change events, scheduled daily report trigger
S3	Lambda deployment packages, CloudFormation templates, daily report CSVs
EC2	t2.micro Flask app for internal reporting dashboard
CloudWatch	Centralized logs, custom metrics, dashboards, alarms
SSM Parameter Store	SecureString for auth token, DB credentials, topic ARNs, configurable thresholds
GitHub + GitHub Actions	Source control, CI/CD pipeline, automated deploy on push to main

Engineering Concepts

- **Infrastructure as Code** — declarative resource management, parameterization, stack decomposition, Change Set discipline
 - **CI/CD** — automated pipeline triggered by code push, packaging, deployment, smoke testing
 - **Token-based API authentication** — Lambda Authorizer pattern, Bearer token validation, authorizer caching, SSM SecureString
 - **Event-driven architecture** — loose coupling via EventBridge and SQS, publisher/subscriber pattern, DLQ for failure capture
 - **Relational data modelling** — normalization, foreign key constraints, index design for query patterns
 - **NoSQL data modelling** — access-pattern-first key design, partition key distribution, GSI tradeoffs
 - **Least-privilege security** — IAM role scoping, resource policies, no credentials in code
 - **Network security** — VPC subnet isolation, security groups as firewalls, private RDS access
 - **Observability** — structured JSON logging, custom CloudWatch metrics, dashboards, actionable alarms
 - **Resilience patterns** — DLQ for failure capture, retry logic, idempotent Lambda handlers
 - **Operational discipline** — environment parameterization, repeatable deployments, documentation
-

Milestones and Review Checkpoints

Calendar reference: Start date 11 August 2026, end date 18 September 2026. Evaluation period 21–30 September 2026. Working days are Monday–Friday.

Milestone 1 — Repository and Design Kickoff

Target Date: 12 August 2026

Goal: Get the GitHub repository up and the high-level design documented on day one. This is a lightweight kickoff checkpoint — detailed design work continues into Milestone 2.

Deliverables due:

- GitHub repository initialized with folder structure and README skeleton
- High-level architecture diagram drafted (services identified, rough data flow, VPC layout)
- CloudFormation stack plan outlined — list of stacks and what each will own

Review checkpoint: Mentor reviews the initial diagram and stack plan for obvious gaps or misunderstandings. This is a directional review, not a final sign-off. Participant notes any open questions to resolve before Milestone 2.

Milestone 2 — Architecture, Design, and CI/CD Bootstrap

Target Date: 19 August 2026

Goal: Complete, detailed system design signed off and core infrastructure deployed via the CI/CD pipeline. No business logic yet — the pipeline and network foundation must be working before any application code is written.

Deliverables due:

- Architecture diagram finalized — all services, data flows, VPC layout, security boundaries, CI/CD pipeline
- RDS schema — all tables, columns, data types, indexes, foreign keys
- DynamoDB key design — partition key, sort key, GSI justification for each access pattern
- Authentication design — Lambda Authorizer flow, SSM parameter names, token rotation plan
- GitHub Actions workflow design — trigger conditions, pipeline steps, secrets strategy
- `network-stack` deployed via GitHub Actions: VPC, public and private subnets, internet gateway, route tables, security groups

- `data-stack` deployed via GitHub Actions: RDS MySQL in private subnet, DynamoDB table, S3 buckets
- IAM roles for all Lambda functions deployed
- SSM Parameter Store entries created for DB credentials and auth token
- Verified: a test Lambda can connect to RDS from within the VPC

Review checkpoint: Mentor reviews the finalized architecture diagram and data model — **no further implementation proceeds until signed off**. GitHub Actions run log reviewed (not the AWS console). Security group rules and IAM policies spot-checked.

Milestone 3 — Authentication Layer and Product Service

Target Date: 26 August 2026

Goal: API authentication working and the first business domain live end-to-end.

Deliverables due:

- `auth-stack` deployed: Lambda Authorizer function, SSM SecureString for token
- Lambda Authorizer attached to all API Gateway endpoints
- Verified: requests without a token return `401`, wrong token returns `401`, correct token returns `200`
- Authorizer cache TTL set and verified
- `api-stack` (partial): API Gateway + product Lambda deployed with Lambda Authorizer on all methods
- RDS schema applied — product and inventory tables created with sample data
- Product CRUD endpoints working: `POST /products`, `GET /products`, `GET /products/{id}`, `PUT /products/{id}`, `DELETE /products/{id}`
- EventBridge rule firing on inventory change events
- SNS low-stock alert triggering and delivering to email when stock falls below threshold
- All Lambda functions logging structured JSON to CloudWatch
- All changes deployed exclusively via GitHub Actions

Review checkpoint: Mentor sends three curl requests live — no token, wrong token, correct token. Full product flow demonstrated via curl — create, retrieve, update stock, receive low-stock email. Authorizer Lambda reviewed for hardcoded values (there must be none).

Milestone 4 — Order Processing Service

Target Date: 2 September 2026

Goal: Core order flow working with failure handling, all endpoints authenticated.

Deliverables due:

- Order placement API: `POST /orders`, `GET /orders/{id}`, `GET /orders?customerId=X` — all behind Lambda Authorizer
- DynamoDB order write confirmed on placement
- Inventory deduction in RDS on order confirmation
- SQS queue receiving orders for async processing
- DLQ receiving and retaining failed orders
- Order confirmation SNS notification delivered to email
- EventBridge publishing `OrderPlaced`, `OrderConfirmed`, `OrderFailed` events
- All changes deployed via GitHub Actions

Review checkpoint: Live demo of placing an authenticated order, observing the SQS message, intentionally causing a processing failure and showing the DLQ message. DynamoDB key design reviewed against the actual query patterns in use.

Milestone 5 — Reporting, EC2 Dashboard, and Observability

Target Date: 9 September 2026

Goal: Operations visibility layer complete — dashboards, alarms, and the EC2 reporting dashboard all live.

Deliverables due:

- Daily report Lambda running on EventBridge schedule and writing CSV to S3
- EC2 Flask dashboard deployed and accessible — shows current product inventory, recent orders, and link to latest S3 report
- `monitoring-stack` deployed: `cloudmart-operations` CloudWatch dashboard with all required widgets
- All four CloudWatch alarms configured and delivering to SNS
- Custom CloudWatch metrics `OrdersPlaced`, `OrdersFailed`, `LowStockEvents` being published from Lambda
- EC2 deployment automated via UserData script or GitHub Actions step

Review checkpoint: Mentor accesses the EC2 dashboard URL live. CloudWatch dashboard reviewed together — all widgets must show data. At least one alarm deliberately triggered and SNS notification received during the review.

Milestone 6 — Hardening, Integration, and Final Submission

Target Date: 17 September 2026

Goal: System complete, hardened, fully documented, and ready for evaluation submission the following day.

Deliverables due:

- Full end-to-end integration verified: authenticated API call → DynamoDB write → RDS inventory deduction → SNS notification → CloudWatch metric incremented
- IAM policy audit complete — all wildcard actions removed, every role scoped to specific resource ARNs
- CloudFormation drift detection run on all stacks — all stacks show IN_SYNC
- All CloudFormation stack updates performed via Change Sets (evidenced in GitHub Actions logs)
- All deliverables finalized and committed to GitHub by EOD 18 September
- System torn down completely and redeployed from scratch by running the GitHub Actions pipeline — must succeed without any manual steps
- 20-minute walkthrough demo prepared

Review checkpoint: Mentor-led final walkthrough. Participant demonstrates: an authenticated API call end-to-end, a code push triggering the CI/CD pipeline, the live CloudWatch dashboard, one CloudWatch alarm firing, and the architecture diagram mapped to the running system. Participant explains at least two design decisions and one thing they would change with hindsight.

Mentor Engagement Model

Availability

- Mentor is available for a **scheduled 45-minute check-in at each milestone boundary** — 6 sessions total

- Additional async questions via Slack/Teams are welcome — expected response within the same working day
- Mentor does not debug code or write CloudFormation on behalf of participants — questions must be framed as: what I am trying to do, what I tried, what the error says, and what my hypothesis is

What Mentor Does Not Do

- Write code, CloudFormation templates, or GitHub Actions workflows for participants
- Make architecture decisions — decisions can be challenged and questioned, but not made for participants
- Debug Lambda errors or CloudFormation stack failures — participants use CloudWatch Logs and CloudFormation Events first, then escalate with a structured problem description

Escalation

If blocked for more than 2 hours on a single issue, post a structured block message:

1. What I am trying to do
2. What I tried
3. The exact error message or unexpected behavior
4. My current hypothesis about the cause

Mentor responds with a directional hint. Not a solution.

Evaluation Criteria and Success Measures

Scoring Categories

Category	What Is Assessed
Architecture Quality	Correctness of design, appropriate service choices, VPC layout, security boundary design, alignment between diagram and built system
Infrastructure as Code	CloudFormation completeness, parameterization, stack decomposition, Change Set usage, no manually created resources
CI/CD Pipeline	GitHub Actions workflow functionality, deployment on push, Change Set-based updates, secrets handling, smoke test

Category	What Is Assessed
Authentication	Lambda Authorizer correctness, SSM-backed token, cache configuration, 401 on invalid/missing token
Application Code Quality	Clean Python, no hardcoded config, proper error handling, structured JSON logging, idempotent handlers
Observability	CloudWatch dashboard completeness, custom metrics, actionable alarms with SNS delivery
Documentation and Communication	Architecture diagram accuracy, data model documentation, runbook completeness
Demo and Presentation	Clarity of explanation, ability to justify design decisions, live system behavior, handling of evaluator questions

Success Measures

A strong submission will satisfy all of the following:

- The system deploys successfully by running the GitHub Actions pipeline into a fresh account — no manual steps required beyond configuring GitHub secrets
- All API endpoints reject unauthenticated requests with `401`; authenticated requests return correct responses
- An order placed via the API results in a DynamoDB write, RDS inventory deduction, and SNS notification — verified live
- The `cloudmart-operations` CloudWatch dashboard is populated with live data across all required widgets
- At least one CloudWatch alarm fires and delivers an SNS notification during the evaluation period
- No IAM role contains a wildcard action (`*`) on any resource
- No credentials, ARNs, or account IDs are hardcoded in code or CloudFormation templates
- All CloudFormation stacks show `IN_SYNC` in drift detection at submission time
- The GitHub repository has a complete README, architecture diagram, data model doc, and commit history distributed meaningfully across the project timeline
- OIDC-based GitHub Actions authentication, CloudFormation Change Sets used for every stack update, and DLQ alarms triggering custom metric publishing are indicators of above-average work

What Will Cause Failure Regardless of Score

- Any AWS credentials committed to GitHub at any point — even in a deleted commit or a branch
 - RDS instance accessible from the public internet (security group allows `0.0.0.0/0` on port 3306)
 - Any CloudFormation stack or AWS resource created or modified manually via the console after Milestone 2
 - Authentication bypassed or missing on any API endpoint
 - System requires manual steps not in the runbook to deploy
 - No meaningful commit history (single bulk commit or commits only at milestone deadlines)
-

Evaluation Phase — 21 September to 30 September 2026

The evaluation window runs from 21 September to 30 September 2026. Participants submit on 18 September and evaluators review independently before the demo.

Submission Requirements (by EOD 18 September 2026)

- GitHub repository finalized — no commits after EOD 18 September except to fix broken links or typos in documentation
- All deliverables committed and accessible: architecture diagram, data model doc, runbook, README
- GitHub Actions pipeline in a passing state on the `main` branch — the last pipeline run must be green
- A brief submission note added to the README confirming the system is deployed and the invoke URL is accessible

What Evaluators Will Do

- Review the GitHub repository, CloudFormation templates, IAM policies, and GitHub Actions workflows independently
- Attempt a fresh deployment following only the documented runbook — no participant assistance
- Send unauthenticated and authenticated API requests across all endpoints
- Inspect every IAM role for wildcard actions and verify RDS is not publicly accessible
- Check CloudFormation drift detection on all stacks
- Review GitHub commit history for distribution across the project timeline and check for any credential exposure

A note on cost: All services in this project operate within AWS Free Tier limits when used as specified. RDS `db.t3.micro` single-AZ and EC2 `t2.micro` are free for 12 months on new accounts. DynamoDB, Lambda, API Gateway, SQS, SNS, EventBridge, and S3 all have permanent free tiers sufficient for this workload. Do not create any customer-managed KMS keys — use AWS-managed keys (`aws/ssm`, `aws/rds`) everywhere. The primary cost risk is leaving RDS and EC2 running after the project ends. The deployment runbook must include a full teardown section, and participants are responsible for deleting all resources at project completion.